Week 1 Full Notes: AI Foundations

Goal of the week: understand what LLMs are and how they work, then build a working chatbot that calls one from Java.

Day 1: The AI Landscape

1. The nested terms

```
AI  >  Machine Learning  >  Deep Learning  >  LLMs
(each term is a subset of the one before it)
```

Term	Meaning	Example
AI	Any system that performs tasks that normally need human intelligence	Chess engine, chatbot, route planner
ML	Systems that learn patterns from data instead of hand-written rules	Spam filter learning from labeled emails
Deep learning	ML using neural networks with many layers	Image recognition, speech-to-text
LLM	A deep learning model trained on huge amounts of text to predict the next token	GPT, Claude, Gemini, Llama

Rules vs. learning: in traditional programming, you write rules and the computer applies them to data to get answers. In ML, you give the computer data and answers, and it learns the rules.

2. Predictive vs. generative AI

- **Predictive AI** outputs a label or number: fraud or not fraud, tomorrow's demand, a credit score.
- **Generative AI** creates new content: text, code, images, audio, video.

3. Roles

Role	Does	Skills
Data scientist	Analyzes data, builds models	Statistics, Python
ML engineer	Trains, tunes and deploys models	Math, PyTorch, GPUs
AI engineer	Builds products on top of existing models (APIs, RAG, agents)	**Backend engineering, prompting, system design**

You are heading toward the third role. Your Java and Spring experience transfers directly.

4. Where LLMs fit in a backend

An LLM behaves like a **powerful but unreliable external service**. It takes text in and gives text out, it is slow, it costs money per call, and its answers can be wrong. Most of AI engineering is making that service dependable.

Day 1 Summary

- AI contains ML, which contains DL, which contains LLMs.
- Predictive AI outputs labels. Generative AI creates content.
- AI engineers use models. ML engineers build them.

Exercise: draw the nested diagram and give one example per level. Write down one feature in a past project of yours that could use an LLM.

Day 2: ML Basics

1. Vocabulary

- **Dataset:** a collection of examples.
- **Features:** the inputs. For a house, these are area, bedrooms and location.
- **Label (target):** the answer to predict, such as the price.
- **Model:** the mathematical function that maps features to a prediction.
- **Training:** adjusting the model's internal numbers (parameters) to fit the data.
- **Inference:** using the trained model on new data. This is what your backend will do.

2. Types of learning

Type	Data	Task examples
Supervised	Labeled	Classification (spam or not), regression (predict price)
Unsupervised	Unlabeled	Clustering customers, anomaly detection
Reinforcement	Rewards from an environment	Game playing, and part of how chat models are tuned

LLMs are first trained with a form of self-supervised learning. The "label" is simply the next word in the text.

3. Overfitting and underfitting

- **Overfitting:** the model memorizes training data and does badly on new data. It is like a student who memorized past papers but can't solve new questions.
- **Underfitting:** the model is too simple to capture the pattern.
- **Fixes for overfitting:** more data, a simpler model, regularization, and early stopping.

4. Train/test split

Never judge a model on data it was trained on. Split the data, for example 80% to train and 20% to test. This habit returns in Week 4 when you evaluate RAG.

5. Hands-on: scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)          # features, labels
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)                # training

preds = model.predict(X_test)              # inference
print("Accuracy:", accuracy_score(y_test, preds))
```

Setup: `pip install scikit-learn`

Day 2 Summary

- Features go in and a label comes out. Training learns and inference uses.
- Always test on unseen data.
- Overfitting means memorizing instead of learning.

Exercise: run the code, then change `test_size` to 0.5 and watch how accuracy changes.

Day 3: Neural Network Intuition

1. The neuron

A neuron does three things:

1. Multiplies each input by a **weight**.
2. Adds them together with a **bias**.
3. Passes the result through an **activation function** (such as ReLU) that adds non-linearity.

```
output = activation( w1*x1 + w2*x2 + ... + bias )
```

2. Layers

- **Input layer:** receives the raw numbers (for example pixel values).
- **Hidden layers:** learn increasingly abstract patterns. Early layers might detect edges, and later ones shapes.
- **Output layer:** gives the prediction.
- "Deep" means many hidden layers.

3. How a network learns (the training loop)

1. **Forward pass:** feed in an example and get a prediction.
2. **Loss:** measure how wrong the prediction is.
3. **Backpropagation:** calculate how much each weight contributed to the error.
4. **Gradient descent:** adjust each weight a small step in the direction that reduces the error. The step size is the **learning rate**.
5. Repeat over millions of examples.

Picture a ball rolling down a hilly landscape to find the lowest point. The height is the loss, and each weight is a direction you can move in.

4. What the numbers mean

- A model's knowledge is stored in its **parameters** (weights and biases).
- "7B" or "70B" means 7 or 70 billion parameters. More parameters generally means more capability but also more memory, cost and latency.

5. Why this matters to you

You won't train networks as an AI engineer, but you need to know that:

- Models are fixed once trained. They don't learn from your API calls.
- Their behavior comes from training data, which is why they have a knowledge cutoff.

Day 3 Summary

- Neuron = weighted sum + bias + activation.
- Training = predict, measure error, backpropagate, adjust, repeat.
- Parameters are the stored knowledge.

Exercise: after watching the 3Blue1Brown videos, explain in 5 lines, without jargon, how a network learns to recognize a handwritten digit.

Day 4: Transformers and LLMs

1. Tokens

- LLMs process **tokens**, which are chunks of text: whole words, parts of words, or punctuation.
- Rough guide for English: **1 token = about 4 characters = about 3/4 of a word**.
- Code, numbers and non-English text usually use more tokens per word.
- Example split: "Unbelievable" might become Un + believ + able.
- Everything is measured in tokens: **cost, speed, and limits**.

2. Embeddings (first look)

Each token is converted into a list of numbers (a vector) that captures its meaning. Similar meanings end up close together. You will use this heavily in Week 3.

3. Attention

- For each token, the model works out **which other tokens in the text matter most**.
- Example: in "The trophy didn't fit in the suitcase because **it** was too big", attention helps connect "it" to "trophy".
- The **Transformer** architecture stacks many attention layers. Nearly all modern LLMs use it.

4. How an LLM generates text

```
Prompt -> model predicts probabilities for the next token
       -> one token is picked (sampling)
       -> it is appended to the text
       -> repeat until done or max tokens reached
```

The model writes one token at a time. This is why responses can be **streamed** (Week 2).

5. Training stages (simplified)

1. **Pre-training:** learns language and world knowledge by predicting next tokens on enormous text.
2. **Instruction tuning / alignment:** trained to follow instructions and be helpful and safe, which turns a text predictor into a chat assistant.

6. Context window

- The maximum number of tokens the model can handle in one call: **input plus output combined**.
- **The model has no memory between calls.** Each call is independent. To "remember", you must send the earlier conversation again every time.
- A longer prompt means higher cost and slower responses.

7. Why LLMs hallucinate

- They are trained to produce *plausible* text, not to look up verified facts.
- They don't know what they don't know, so they may fill gaps confidently.
- Their knowledge stops at a training cutoff date.
- They know nothing about your private data.

Remedies you'll learn: RAG (give it real documents), tool calling (let it fetch live data), validation, and clear instructions to say "I don't know".

8. Strengths and weaknesses

Good at	Weak at
Summarizing, rewriting, classifying	Exact arithmetic
Extracting data from text	Up-to-date facts without tools
Writing and explaining code	Guaranteed consistency
Translation, Q&A over provided text	Knowing your private data

Day 4 Summary

- Text becomes tokens, and tokens become vectors.
- Attention lets tokens look at each other.
- The model predicts one token at a time and is stateless.
- Hallucination comes from "plausible, not verified".

Exercise: paste an English paragraph, a Java method and a sentence in your mother tongue into a tokenizer playground. Record the token count for each and note which is the most "expensive".

Day 5: LLM Parameters

1. Sampling parameters

Parameter	Effect	Guidance
temperature	Controls randomness. 0 is nearly deterministic, higher values are more varied	0 to 0.3 for extraction, classification and code. 0.7 to 1 for brainstorming and creative writing
top-p	Samples only from the smallest set of tokens whose probabilities add up to p	Adjust either this or temperature, not both
max tokens	Hard cap on response length	Protects cost, but too small cuts answers mid-sentence
stop sequences	Text that makes the model stop	Useful for structured output

Note that even at temperature 0, outputs may not be perfectly identical every time.

2. Message roles

```
[
  {"role": "user", "content": "What is Spring Boot?"},
  {"role": "assistant", "content": "Spring Boot is ..."},
  {"role": "user", "content": "Show a REST controller example."}
]
```

- **System prompt:** sets role, tone, rules and format. In the Anthropic API it is a separate `system` field. In the OpenAI API it is a message with role `system`.
- **User:** what the human asks.
- **Assistant:** the model's previous replies, which you resend as history.

3. Writing a good system prompt

```
You are a senior Java mentor.
- Answer in simple English with short paragraphs.
- Include a code example when relevant.
- If you are not sure, say so instead of guessing.
```

4. Pricing

- Providers charge **per million tokens**, with separate rates for input and output (output is usually more expensive).
- `cost = (input_tokens x input_rate) + (output_tokens x output_rate)`
- Your whole conversation history counts as input on every call, so long chats get expensive.
- Smaller models cost much less and are often enough for simple tasks.
- Always check the provider's current pricing page.

5. Choosing a model

Think in three tiers: a small and fast model, a balanced model, and a large and powerful model. Start with the balanced one, then move simple tasks down to the small model to save money.

Day 5 Summary

- Low temperature gives consistent output. High temperature gives creative output.
- The system prompt sets behavior. History must be resent.
- Cost = tokens in + tokens out.

Exercise: send the same prompt ("Write a tagline for a coffee shop") 3 times at temperature 0 and 3 times at temperature 1. Compare the results and note the difference.

Day 6: First API Calls

1. Setup

1. Create an account with a provider and generate an **API key**.
2. Store it in an environment variable. **Never hard-code it or commit it to Git.**

```
export ANTHROPIC_API_KEY="your-key-here"      # Linux/Mac
setx ANTHROPIC_API_KEY "your-key-here"        # Windows
```

2. First call with curl

```
curl https://api.anthropic.com/v1/messages \
  -H "content-type: application/json" \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -d '{
    "model": "claude-sonnet-5-5",
    "max_tokens": 300,
    "system": "You are a concise Java mentor.",
    "messages": [{"role": "user", "content": "What is a token?"}]
  }'
```

Check the provider docs for the current model names and version header.

3. Reading the response

```
{
  "content": [ {"type": "text", "text": "A token is ..."} ],
  "stop_reason": "end_turn",
  "usage": { "input_tokens": 25, "output_tokens": 80 }
}
```

- `content[0].text` is the answer.
- `stop_reason` tells you why it stopped. `max_tokens` means it was cut off.
- `usage` is what you use to track cost.

4. Call from Java with HttpClient

```
import java.net.URI;
import java.net.http.*;

public class FirstCall {
    public static void main(String[] args) throws Exception {
        String apiKey = System.getenv("ANTHROPIC_API_KEY");
        String body = """
            {
              "model": "claude-sonnet-5-5",
              "max_tokens": 300,
              "system": "You are a concise Java mentor.",
              "messages": [
                {"role": "user", "content": "Explain what a token is."}
              ]
            }
            """;

        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://api.anthropic.com/v1/messages"))
            .header("content-type", "application/json")
            .header("x-api-key", apiKey)
            .header("anthropic-version", "2023-06-01")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        HttpResponse<String> res = HttpClient.newHttpClient()
            .send(req, HttpResponse.BodyHandlers.ofString());

        System.out.println(res.statusCode());
        System.out.println(res.body());
    }
}
```

5. Parsing with Jackson

Maven dependency: `com.fasterxml.jackson.core:jackson-databind`

```
ObjectMapper mapper = new ObjectMapper();
JsonNode root = mapper.readTree(res.body());
String text = root.path("content").get(0).path("text").asText();
int in  = root.path("usage").path("input_tokens").asInt();
int out = root.path("usage").path("output_tokens").asInt();
System.out.println(text + "\n(tokens: " + in + " in, " + out + " out)");
```

6. Common errors

Status	Meaning	What to do
400	Bad request (invalid JSON or parameters)	Check the body
401	Invalid or missing API key	Check the env variable
429	Rate limit	Wait and retry with backoff
5xx / 529	Provider-side problem or overload	Retry later

Day 6 Summary

- An LLM API is just an HTTPS POST with JSON.
- Read `content`, `stop_reason` and `usage`.
- Keep keys in environment variables.

Exercise: run the curl command, then the Java version. Parse and print only the answer text and token counts.

Day 7: Project 1, CLI Chatbot in Java

1. Requirements

- A loop that reads input until the user types `exit`.
- Sends the **full history** on every call (the model is stateless).
- Uses a **system prompt**.
- Prints the reply and the token usage, plus running totals.
- Commands: `/reset` to clear history.
- Friendly handling of errors.

2. Structure

```
chatbot/
 |-- Message.java    (record: role, content)
 |-- LlmClient.java  (HTTP call and JSON parsing)
 |-- ChatBot.java    (main loop and history)
```

3. Code

Message.java

```
public record Message(String role, String content) {}
```

LlmClient.java

```
import com.fasterxml.jackson.databind.*;
import java.net.URI;
import java.net.http.*;
import java.util.*;

public class LlmClient {
    public record Reply(String text, int inputTokens, int outputTokens) {}

    private static final URI URL = URI.create("https://api.anthropic.com/v1/messages");
    private final HttpClient http = HttpClient.newHttpClient();
    private final ObjectMapper mapper = new ObjectMapper();
    private final String apiKey = System.getenv("ANTHROPIC_API_KEY");
    private final String model;

    public LlmClient(String model) {
        if (apiKey == null || apiKey.isBlank())
            throw new IllegalStateException("Set ANTHROPIC_API_KEY first");
        this.model = model;
    }

    public Reply chat(String system, List<Message> history) throws Exception {
        Map<String, Object> body = Map.of(
            "model", model,
            "max_tokens", 500,
            "system", system,
            "messages", history);

        HttpRequest req = HttpRequest.newBuilder().uri(URL)
            .header("content-type", "application/json")
            .header("x-api-key", apiKey)
            .header("anthropic-version", "2023-06-01")
            .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(body)))
            .build();

        HttpResponse<String> res = http.send(req, HttpResponse.BodyHandlers.ofString());
        if (res.statusCode() != 200)
            throw new RuntimeException("API error " + res.statusCode() + ": " + res.body());

        JsonNode root = mapper.readTree(res.body());
        return new Reply(
            root.path("content").get(0).path("text").asText(),
            root.path("usage").path("input_tokens").asInt(),
            root.path("usage").path("output_tokens").asInt());
    }
}
```

ChatBot.java

```
import java.util.*;

public class ChatBot {
    public static void main(String[] args) {
        LlmClient client = new LlmClient("claude-sonnet-5-5");
        List<Message> history = new ArrayList<>();
        String system = "You are a friendly Java mentor. Keep answers short and clear.";
        Scanner in = new Scanner(System.in);
        int totalIn = 0, totalOut = 0;

        System.out.println("Chatbot ready. Type 'exit' to quit, '/reset' to clear.");
        while (true) {
            System.out.print("You: ");
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            if (line.equalsIgnoreCase("exit")) break;
            if (line.equals("/reset")) {
                history.clear();
                System.out.println("History cleared.");
                continue;
            }

            history.add(new Message("user", line));
            try {
                LlmClient.Reply r = client.chat(system, history);
                history.add(new Message("assistant", r.text()));
                totalIn += r.inputTokens();
                totalOut += r.outputTokens();
                System.out.println("Bot: " + r.text());
                System.out.printf("[tokens this turn: %d in / %d out | total: %d in / %d out]%n",
                    r.inputTokens(), r.outputTokens(), totalIn, totalOut);
            } catch (Exception e) {
                history.removeLast();   // don't keep the failed message
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

4. Test it

1. Say: "My name is Ravi." Then ask: "What is my name?" It should answer correctly because you resend the history.
2. Use `/reset` and ask again. It should no longer know.
3. Watch input tokens grow each turn. This shows why long chats cost more.

5. Stretch goals

- Keep only the last N messages to limit tokens.
- Add `/system <text>` to change the persona.
- Retry once on a 429 with a short delay.
- Print an estimated cost using the current per-token rates.

Week 1 Glossary

Term	Meaning
Token	A chunk of text the model reads and writes
Context window	Max tokens per call (input + output)
Parameter	A learned number inside the model
Inference	Running a trained model
Temperature	Randomness setting
System prompt	Instructions that set model behavior
Hallucination	Confident but incorrect output
Stateless	The model remembers nothing between calls

Self-Test Questions

1. What is the difference between AI, ML and an LLM?
2. What is the difference between training and inference?
3. Why do we split data into train and test sets?
4. What is a token, and why does it matter for cost?
5. Why must you resend history on every API call?
6. When would you use temperature 0 vs. 1?
7. Why do LLMs hallucinate, and what helps?
8. What do you find in the `usage` part of an API response?

Week 1 Checklist

- ☐ Can explain AI vs. ML vs. DL vs. LLM
- ☐ Ran a scikit-learn example
- ☐ Watched the neural network videos and can explain how a network learns
- ☐ Understand tokens, context window and hallucination
- ☐ Know temperature, max tokens and system prompts
- ☐ Made a curl call and a Java call
- ☐ CLI chatbot works and remembers the conversation